Jetpack Compose: Lists, Grids, & Pagers

Muhammad Naveed

December 01, 2025

1. Basic Lists & Scrolling

Understanding Row, Column, and Scroll Modifiers

Standard List Containers

Column & Row

The standard Column and Row composables are the simplest way to arrange items. They render all content immediately, making them suitable only for small, static datasets.

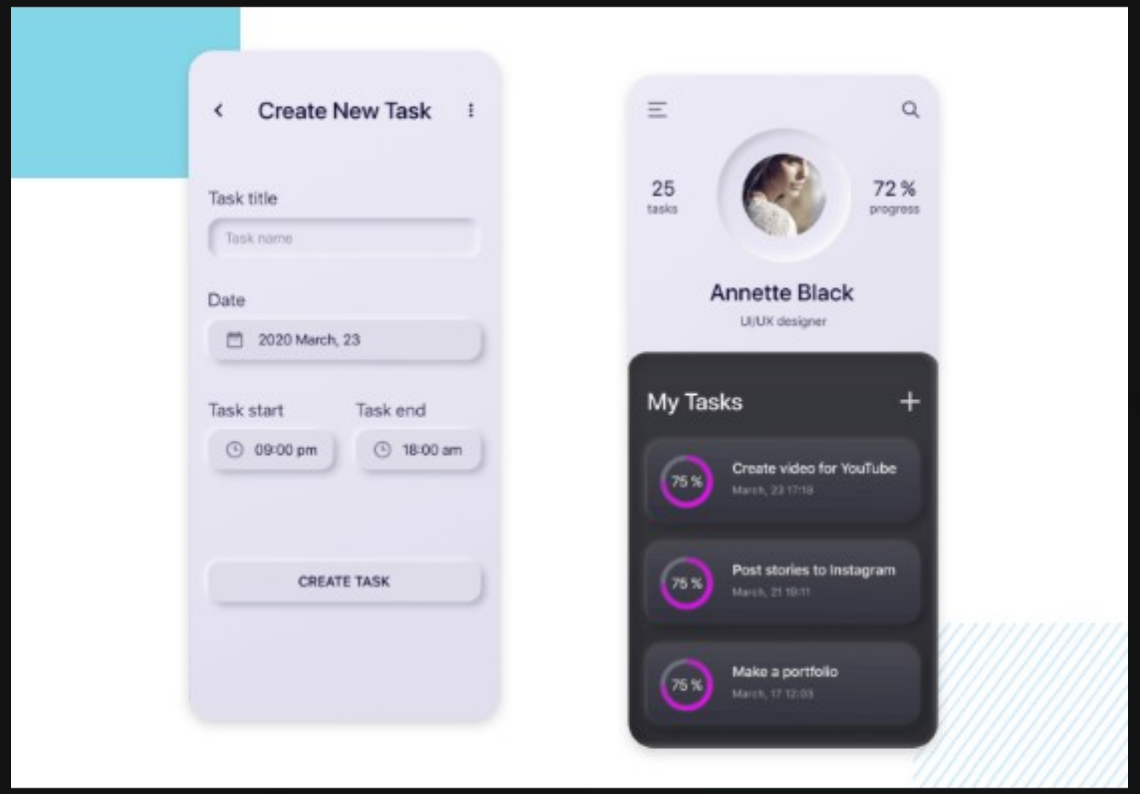
Enabling Scroll

By default, these layouts do not scroll. You must explicitly attach a ScrollState using the .verticalScroll() or .horizontalScroll() modifiers to enable user interaction.

Programmatic Scrolling

Controlling Position

- **ScrollState:** Obtain a reference using rememberScrollState() to monitor and control scroll position.
- **Coroutines:** Scrolling functions like animateScrollTo() are suspend functions. They must be launched within a CoroutineScope.
- **Use Case:** Implementing "Scroll to Top" or "Scroll to End" buttons for quick navigation in long content.



2. Lazy Lists

Optimizing Performance with LazyColumn & LazyRow

The "Lazy" Advantage

Efficiency First

Unlike standard columns, LazyColumn only composes and renders items that are currently visible on screen. This is essential for lists with hundreds or thousands of items to prevent memory crashes.

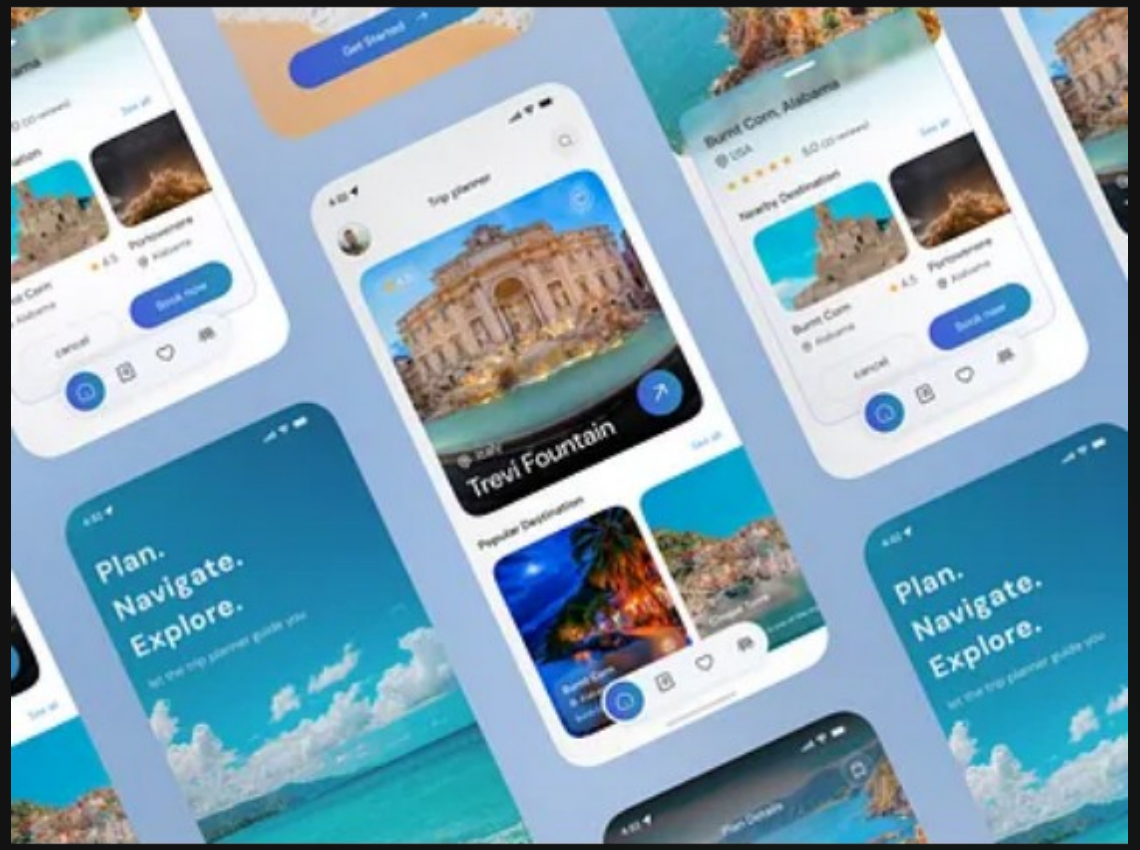
Implementation

Instead of a loop, use the items() DSL. You can pass arrays or lists directly. This separation of data and UI allows for dynamic updates and cleaner code architecture.

Rich Content & Images

Async Image Loading

- **Coil Library:** Use rememberAsyncImagePainter() to load images from URLs asynchronously. It handles caching, downsampling, and decoding automatically.
- **Card Layouts:** Wrap list items in Card composables to provide elevation, rounded corners, and a distinct surface for content like text and images.
- **Interactivity:** Add .clickable {} modifiers to handle user taps, such as showing details or navigation.

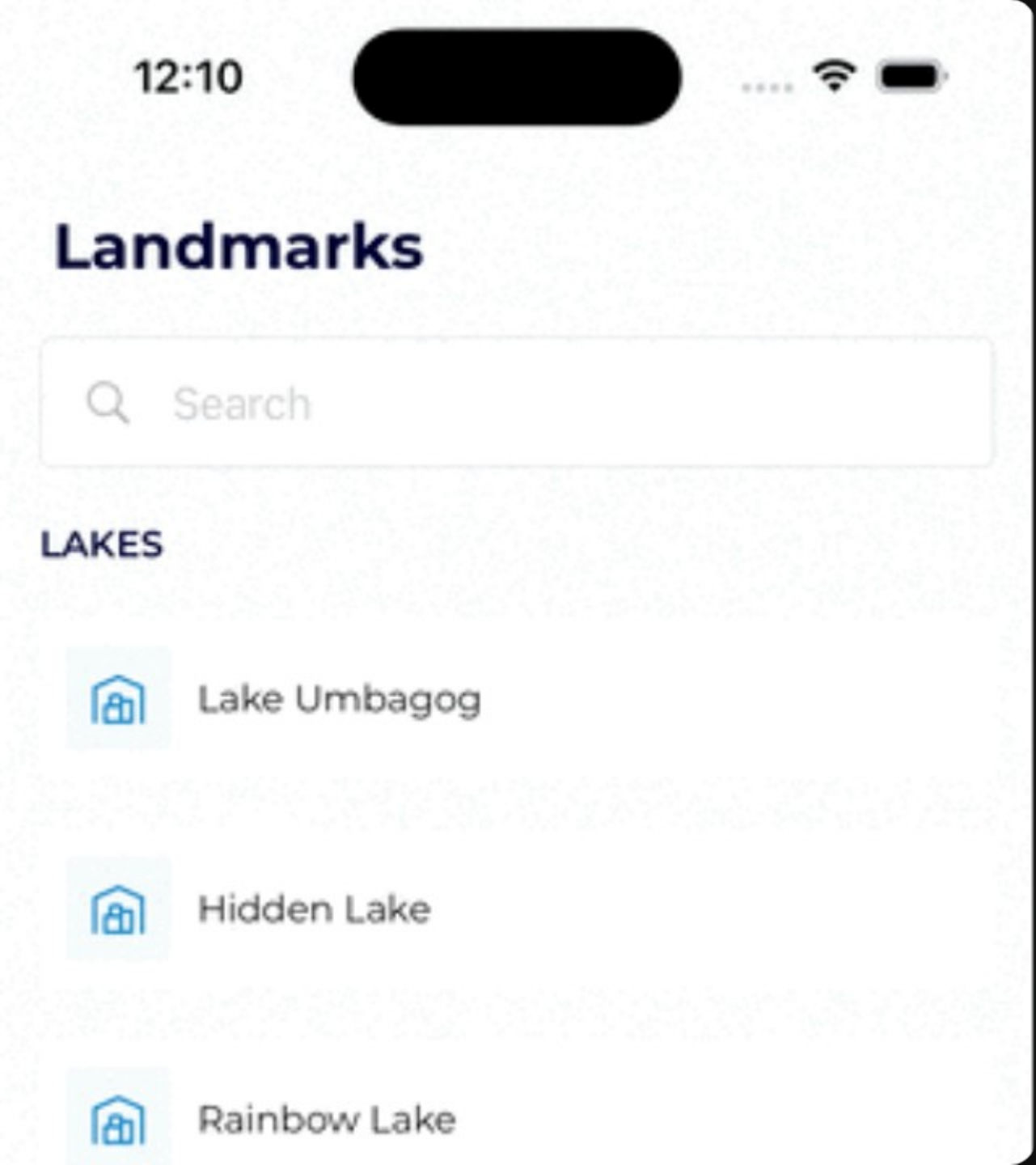


Sticky Headers

Organized Grouping

For categorized data, use the stickyHeader function within a Lazy list. As the user scrolls, the header for the current section remains "stuck" to the top of the view until the next section pushes it away.

This requires grouping your data first (e.g., using Kotlin's groupBy) and iterating through the groups in your list definition.



Scroll Detection & State

Observing State

Use rememberLazyListState() to track the list's status. Properties like firstVisibleItemIndex let you know exactly where the user is in the list.

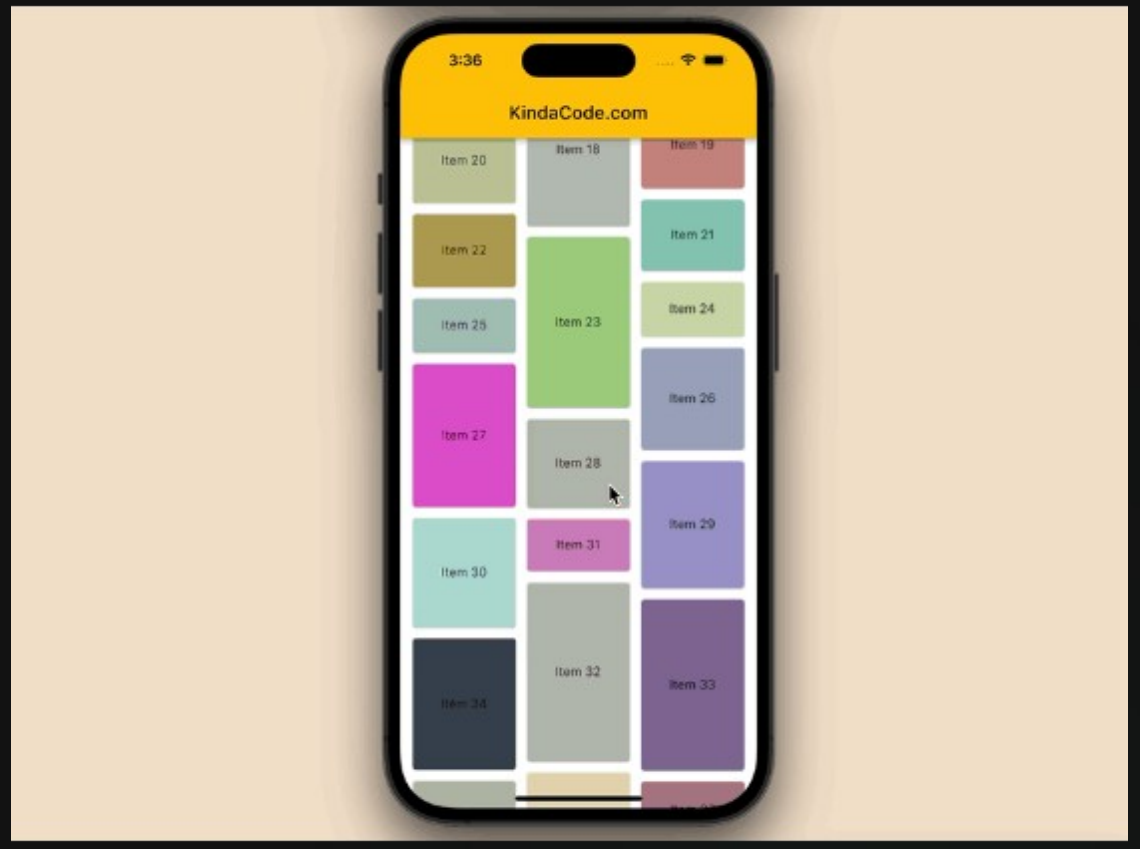
Derived State

To avoid unnecessary recompositions, wrap state calculations in derivedStateOf. For example, show a "Scroll to Top" button only when the user has scrolled past the 5th item.

Staggered Grids

Beyond Uniform Grids

- **The Problem:** Standard grids require uniform cell sizes. This wastes space if items vary significantly in height or width.
- **The Solution:** LazyVerticalStaggeredGrid allows items to flow naturally into columns, creating a "masonry" look similar to Pinterest.
- **Configuration:** Use StaggeredGridCells.Fixed(n) or .Adaptive(minSize) to control the column count based on screen size.



Swipeable Pagers

HorizontalPager

Perfect for image carousels or onboarding screens. Users swipe left or right to navigate between distinct pages of content.

VerticalPager

Creates a full-screen scrolling experience similar to TikTok or YouTube Shorts, snapping to each page as the user swipes.

PagerState

Just like lists, pagers have state. You can programmatically scroll to a page or add indicators to show the current page index.

Questions?

Reviewing Lists, Grids, and Pagers in Compose

Happy Coding!